

Professional Frontend Interview Handbook

**HTML • CSS • JavaScript • React • Machine Coding • Frontend
System Design**

HTML Mastery

Important interview areas:

- Semantic HTML
- Accessibility
- SEO
- Forms
- Media Tags
- Web Storage
- Cookies
- Meta Tags
- Iframe
- Canvas

CSS Mastery

Important interview areas:

- Box Model
- Specificity
- Flexbox
- Grid
- Positioning
- Animations
- Responsive Design
- BEM
- Selectors
- Performance

JavaScript Theory

Important interview areas:

- Execution Context
- Call Stack
- Hoisting
- Closures
- Scope
- Prototype
- Inheritance
- Event Loop
- Promises
- Async/Await
- Currying
- Memoization
- Debounce
- Throttle
- Polyfills

JavaScript Coding

Important interview areas:

- Reverse String
- Flatten Array
- Deep Clone
- Group By
- Custom Map
- Custom Filter
- Custom Reduce
- LRU Cache
- Promise.all
- Event Emitter

React

Important interview areas:

- Virtual DOM
- Reconciliation
- Hooks
- useEffect
- useMemo
- useCallback
- Context API
- Redux
- Performance Optimization
- Code Splitting

Machine Coding

Important interview areas:

- Todo App
- Kanban Board
- Infinite Scroll
- Autocomplete
- Shopping Cart
- Movie Search
- Pagination
- File Explorer
- Chat UI
- Dashboard

Frontend System Design

Important interview areas:

- URL Shortener UI
- YouTube UI
- WhatsApp Web
- Instagram Feed
- Google Docs
- Netflix Home Page
- E-commerce Architecture

300 Interview Questions

1. What is semantic HTML and why is it important?
2. Difference between id and class?
3. How does CSS specificity work?
4. Flexbox vs Grid?
5. Explain event bubbling.
6. Explain event capturing.
7. What is a closure?
8. What is hoisting?
9. Difference between var, let and const?
10. Explain the JavaScript event loop.
11. What are promises?
12. What is async/await?
13. Explain prototype inheritance.
14. What is currying?
15. What is memoization?
16. What is debounce?
17. What is throttle?
18. What is Virtual DOM?
19. How does React reconciliation work?
20. What are React hooks?
21. Difference between useMemo and useCallback?
22. What is Context API?
23. Controlled vs uncontrolled components?
24. How do you optimize React apps?
25. Explain code splitting.
26. Design an autocomplete component.
27. Build an infinite scroll component.
28. Design a shopping cart.
29. How would you build Netflix UI?
30. Explain frontend caching strategies.
31. What is semantic HTML and why is it important?
32. Difference between id and class?
33. How does CSS specificity work?
34. Flexbox vs Grid?
35. Explain event bubbling.
36. Explain event capturing.

37. What is a closure?
38. What is hoisting?
39. Difference between var, let and const?
40. Explain the JavaScript event loop.
41. What are promises?
42. What is async/await?
43. Explain prototype inheritance.
44. What is currying?
45. What is memoization?
46. What is debounce?
47. What is throttle?
48. What is Virtual DOM?
49. How does React reconciliation work?
50. What are React hooks?
51. Difference between useMemo and useCallback?
52. What is Context API?
53. Controlled vs uncontrolled components?
54. How do you optimize React apps?
55. Explain code splitting.
56. Design an autocomplete component.
57. Build an infinite scroll component.
58. Design a shopping cart.
59. How would you build Netflix UI?
60. Explain frontend caching strategies.
61. What is semantic HTML and why is it important?
62. Difference between id and class?
63. How does CSS specificity work?
64. Flexbox vs Grid?
65. Explain event bubbling.
66. Explain event capturing.
67. What is a closure?
68. What is hoisting?
69. Difference between var, let and const?
70. Explain the JavaScript event loop.
71. What are promises?
72. What is async/await?
73. Explain prototype inheritance.
74. What is currying?

75. What is memoization?
76. What is debounce?
77. What is throttle?
78. What is Virtual DOM?
79. How does React reconciliation work?
80. What are React hooks?
81. Difference between useMemo and useCallback?
82. What is Context API?
83. Controlled vs uncontrolled components?
84. How do you optimize React apps?
85. Explain code splitting.
86. Design an autocomplete component.
87. Build an infinite scroll component.
88. Design a shopping cart.
89. How would you build Netflix UI?
90. Explain frontend caching strategies.
91. What is semantic HTML and why is it important?
92. Difference between id and class?
93. How does CSS specificity work?
94. Flexbox vs Grid?
95. Explain event bubbling.
96. Explain event capturing.
97. What is a closure?
98. What is hoisting?
99. Difference between var, let and const?
100. Explain the JavaScript event loop.
101. What are promises?
102. What is async/await?
103. Explain prototype inheritance.
104. What is currying?
105. What is memoization?
106. What is debounce?
107. What is throttle?
108. What is Virtual DOM?
109. How does React reconciliation work?
110. What are React hooks?
111. Difference between useMemo and useCallback?
112. What is Context API?

113. Controlled vs uncontrolled components?
114. How do you optimize React apps?
115. Explain code splitting.
116. Design an autocomplete component.
117. Build an infinite scroll component.
118. Design a shopping cart.
119. How would you build Netflix UI?
120. Explain frontend caching strategies.
121. What is semantic HTML and why is it important?
122. Difference between id and class?
123. How does CSS specificity work?
124. Flexbox vs Grid?
125. Explain event bubbling.
126. Explain event capturing.
127. What is a closure?
128. What is hoisting?
129. Difference between var, let and const?
130. Explain the JavaScript event loop.
131. What are promises?
132. What is async/await?
133. Explain prototype inheritance.
134. What is currying?
135. What is memoization?
136. What is debounce?
137. What is throttle?
138. What is Virtual DOM?
139. How does React reconciliation work?
140. What are React hooks?
141. Difference between useMemo and useCallback?
142. What is Context API?
143. Controlled vs uncontrolled components?
144. How do you optimize React apps?
145. Explain code splitting.
146. Design an autocomplete component.
147. Build an infinite scroll component.
148. Design a shopping cart.
149. How would you build Netflix UI?
150. Explain frontend caching strategies.

151. What is semantic HTML and why is it important?
152. Difference between id and class?
153. How does CSS specificity work?
154. Flexbox vs Grid?
155. Explain event bubbling.
156. Explain event capturing.
157. What is a closure?
158. What is hoisting?
159. Difference between var, let and const?
160. Explain the JavaScript event loop.
161. What are promises?
162. What is async/await?
163. Explain prototype inheritance.
164. What is currying?
165. What is memoization?
166. What is debounce?
167. What is throttle?
168. What is Virtual DOM?
169. How does React reconciliation work?
170. What are React hooks?
171. Difference between useMemo and useCallback?
172. What is Context API?
173. Controlled vs uncontrolled components?
174. How do you optimize React apps?
175. Explain code splitting.
176. Design an autocomplete component.
177. Build an infinite scroll component.
178. Design a shopping cart.
179. How would you build Netflix UI?
180. Explain frontend caching strategies.
181. What is semantic HTML and why is it important?
182. Difference between id and class?
183. How does CSS specificity work?
184. Flexbox vs Grid?
185. Explain event bubbling.
186. Explain event capturing.
187. What is a closure?
188. What is hoisting?

189. Difference between var, let and const?
190. Explain the JavaScript event loop.
191. What are promises?
192. What is async/await?
193. Explain prototype inheritance.
194. What is currying?
195. What is memoization?
196. What is debounce?
197. What is throttle?
198. What is Virtual DOM?
199. How does React reconciliation work?
200. What are React hooks?
201. Difference between useMemo and useCallback?
202. What is Context API?
203. Controlled vs uncontrolled components?
204. How do you optimize React apps?
205. Explain code splitting.
206. Design an autocomplete component.
207. Build an infinite scroll component.
208. Design a shopping cart.
209. How would you build Netflix UI?
210. Explain frontend caching strategies.
211. What is semantic HTML and why is it important?
212. Difference between id and class?
213. How does CSS specificity work?
214. Flexbox vs Grid?
215. Explain event bubbling.
216. Explain event capturing.
217. What is a closure?
218. What is hoisting?
219. Difference between var, let and const?
220. Explain the JavaScript event loop.
221. What are promises?
222. What is async/await?
223. Explain prototype inheritance.
224. What is currying?
225. What is memoization?
226. What is debounce?

227. What is throttle?

228. What is Virtual DOM?

229. How does React reconciliation work?

230. What are React hooks?

231. Difference between useMemo and useCallback?

232. What is Context API?

233. Controlled vs uncontrolled components?

234. How do you optimize React apps?

235. Explain code splitting.

236. Design an autocomplete component.

237. Build an infinite scroll component.

238. Design a shopping cart.

239. How would you build Netflix UI?

240. Explain frontend caching strategies.

241. What is semantic HTML and why is it important?

242. Difference between id and class?

243. How does CSS specificity work?

244. Flexbox vs Grid?

245. Explain event bubbling.

246. Explain event capturing.

247. What is a closure?

248. What is hoisting?

249. Difference between var, let and const?

250. Explain the JavaScript event loop.

251. What are promises?

252. What is async/await?

253. Explain prototype inheritance.

254. What is currying?

255. What is memoization?

256. What is debounce?

257. What is throttle?

258. What is Virtual DOM?

259. How does React reconciliation work?

260. What are React hooks?

261. Difference between useMemo and useCallback?

262. What is Context API?

263. Controlled vs uncontrolled components?

264. How do you optimize React apps?

265. Explain code splitting.
266. Design an autocomplete component.
267. Build an infinite scroll component.
268. Design a shopping cart.
269. How would you build Netflix UI?
270. Explain frontend caching strategies.
271. What is semantic HTML and why is it important?
272. Difference between id and class?
273. How does CSS specificity work?
274. Flexbox vs Grid?
275. Explain event bubbling.
276. Explain event capturing.
277. What is a closure?
278. What is hoisting?
279. Difference between var, let and const?
280. Explain the JavaScript event loop.
281. What are promises?
282. What is async/await?
283. Explain prototype inheritance.
284. What is currying?
285. What is memoization?
286. What is debounce?
287. What is throttle?
288. What is Virtual DOM?
289. How does React reconciliation work?
290. What are React hooks?
291. Difference between useMemo and useCallback?
292. What is Context API?
293. Controlled vs uncontrolled components?
294. How do you optimize React apps?
295. Explain code splitting.
296. Design an autocomplete component.
297. Build an infinite scroll component.
298. Design a shopping cart.
299. How would you build Netflix UI?
300. Explain frontend caching strategies.

50 Machine Coding Challenges

1. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
2. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
3. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
4. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
5. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
6. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
7. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
8. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
9. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
10. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
11. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
12. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
13. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
14. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
15. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
16. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
17. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
18. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
19. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
20. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
21. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
22. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.

46. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
47. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
48. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
49. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.
50. Build a production-ready frontend challenge with clean architecture, responsive UI, and optimized performance.

5-Day Crash Plan

Day 1: HTML + CSS Fundamentals

Day 2: Advanced CSS + Responsive Design

Day 3: Core JavaScript + Coding Problems

Day 4: React + Hooks + State Management

Day 5: Machine Coding + Mock Interviews + Revision